

Bitácora II

CA0305 - Herramientas de Ciencia de Datos II

Anthony Flores Rojas (C32975)
Andrey González Bastos (C33329)
Randal Picado Bermúdez (C36024)
Leonardo Vega Aragón (C38313)

19 de junio de 2026

1. LINEAR LAYER FORWARD

Se implementa la función `linear_forward(x, w, b)` que calcula la operación de una capa lineal, definida matemáticamente como:

$$Y = XW + B$$

Para que el producto matricial XW tenga sentido, las dimensiones deben ser compatibles. Si $X \in \mathbb{R}^{N \times D}$ y $W \in \mathbb{R}^{D \times M}$, entonces el número de columnas de X debe ser igual al número de filas de W , produciendo:

$$XW \in \mathbb{R}^{N \times M}$$

Cada entrada de la matriz resultante se calcula como:

$$(XW)_{ij} = \sum_{k=1}^D X_{ik} W_{kj}$$

Finalmente se suma el vector de bias $B \in \mathbb{R}^M$, que se distribuye por broadcasting sobre cada fila del resultado:

$$Y_{ij} = \sum_{k=1}^D X_{ik} W_{kj} + B_j$$

2. LINEAR LAYER BACKWARD

Se implementa la función `linear_backward(dout, x, w, b)` que calcula los gradientes de la capa lineal $Y = XW + B$ respecto a cada uno de sus parámetros, usando la regla de la cadena. El gradiente de entrada es $\frac{\partial L}{\partial Y} = \text{dout} \in \mathbb{R}^{N \times D_{out}}$.

Gradiente respecto a X :

Indica cuánto contribuyó cada feature de entrada a la pérdida, y propaga el gradiente hacia la capa anterior:

$$\frac{\partial L}{\partial X} = \text{dout} \cdot W^T \in \mathbb{R}^{N \times D_{in}}$$

Gradiente respecto a W :

Indica cuánto debe cambiar cada peso para reducir la pérdida, y se usa para actualizar W en la optimización:

$$\frac{\partial L}{\partial W} = X^T \cdot \text{dout} \in \mathbb{R}^{D_{in} \times D_{out}}$$

Gradiente respecto a B :

Como B se suma por broadcasting a cada muestra del batch, su gradiente acumula la contribución de todas las muestras:

$$\frac{\partial L}{\partial B_j} = \sum_{i=1}^N \text{dout}_{ij} \in \mathbb{R}^{D_{out}}$$

3. ReLU ACTIVATION FORWARD

Se presenta la 'Rectified Linear Unit', la cual es una de las funciones de activación más usadas en el contexto de redes neuronales, pues les permite a estas estructuras el aprender patrones complejos y aproximar funciones no lineales. En síntesis, permite enseñarle el concepto de no-linealidad a las redes neuronales. Además, su simplicidad hace que sea computacionalmente eficiente (pues, en el fondo, consiste en calcular el máximo dentro de un conjunto de datos).

Se mencionan cuatro aplicaciones. como lo son en el área de redes neuronales convolucionales, redes generadoras y discriminadoras, entre otras.

Sus principales cualidades es que es equivalente a la función identidad para los números positivos, y a la función idénticamente cero para el resto de números, además de que es no lineal y no acotada. Además, permite esparcir los datos al reemplazar varios valores por cero.

4. RELU ACTIVATION BACKWARD

Fundamentación matemática: La propagación inversa mediante RELU (Backpropagation through ReLU) permite estudiar los gradientes fluyen, en dirección opuesta, dentro de la función de activación, lo que permite aprender a la red de sus errores. Este proceso es computacionalmente eficiente y permite localizar neuronas muertas (ejemplares que nunca se activan) y entender la causa de su muerte.

Dado que se conoce el cambio de la pérdida (los errores que comete el modelo) con respecto al output de `relu_forward()`, se puede determinar el cambio de la pérdida con respecto a los tensores analizados (gracias a la regla de la cadena y a que se puede determinar el cambio del output de `relu_forward()` con respecto a los tensores analizados). Con todo lo anterior, y debido a que la pendiente de `relu_forward()` es 1 si $x > 0$ y 0 si $x \leq 0$, entonces se deduce para la i -ésima entrada del vector de pérdidas, su derivada con respecto al tensor es la i -ésima entrada del vector gradiente multiplicado por la i -ésima entrada del vector que registra las pendientes de `relu_forward()` (la cual es o 1 ó 0).

Si la derivada es 1 (el valor de `relu_forward()` asociado era mayor que 0), significa que el gradiente 'pudo devolverse' durante la función de activación. En caso contrario, significa que el gradiente no pudo devolverse (es decir, el modelo no pudo aprender correctamente de sus errores).

5. SIGMOID ACTIVATION

Se implementa la función `sigmoid_ops(x, dout)` que realiza tanto el paso hacia adelante como el paso hacia atrás de la función de activación sigmoide.

La función sigmoide se define como:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Su rango de salida es $(0, 1)$, lo que la hace especialmente útil para interpretar salidas como probabilidades. Tiene forma de S, es simétrica alrededor del punto $(0, 0.5)$ y es diferenciable en todo su dominio.

Para evitar problemas de desbordamiento numérico con valores muy negativos de x , donde e^{-x} puede volverse extremadamente grande, se utiliza una forma equivalente según el signo de la entrada:

$$\sigma(x) = \begin{cases} \frac{1}{1 + e^{-x}} & \text{si } x \geq 0 \\ \frac{e^x}{1 + e^x} & \text{si } x < 0 \end{cases}$$

Ambas expresiones son matemáticamente idénticas, pero la segunda es numéricamente estable para entradas negativas. La derivada de la sigmoide tiene una forma muy conveniente, pues puede calcularse directamente a partir del output del forward sin necesidad de recomputar nada:

$$\sigma'(x) = \sigma(x)(1 - \sigma(x))$$

Aplicando la regla de la cadena, el gradiente que se propaga hacia atrás es:

$$\frac{\partial L}{\partial x} = \text{dout} \cdot \sigma(x)(1 - \sigma(x))$$

Esta función se puede ver en ejemplos del mundo real como en la capa de salida de modelos de clasificación binaria, donde se necesita que la red produzca una probabilidad entre 0 y 1.

6. TANH ACTIVATION

Se implementa la función `tanh_ops(x, dout)` que realiza tanto el paso hacia adelante como el paso hacia atrás de la función de activación tangente hiperbólica.

La función tangente hiperbólica se define como:

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Su rango de salida es $(-1, 1)$, a diferencia de la sigmoide que produce valores en $(0, 1)$. Esto la hace centrada en cero, lo que en la práctica acelera la convergencia durante el entrenamiento pues los gradientes no están sesgados hacia un lado. Al igual que la sigmoide, tiene forma de S y es diferenciable en todo su dominio.

NumPy implementa `np.tanh` de forma optimizada y numéricamente estable, por lo que se usa directamente en el forward. Su derivada también tiene una forma conveniente que se puede calcular a partir del output del forward:

$$\tanh'(x) = 1 - \tanh^2(x)$$

Aplicando la regla de la cadena, el gradiente que se propaga hacia atrás es:

$$\frac{\partial L}{\partial x} = \text{dout} \cdot (1 - \tanh^2(x))$$

7. MSE LOSS FORWARD & BACKWARD

La función de pérdida "Mean squared error" es un estándar en los problemas de regresión. Esta permite medir la distancia promedio entre las predicciones del modelo y los valores reales. Esta misma penaliza los errores "grandes" gracias al término cuadrático.

$$L(\hat{y}_1, \dots, \hat{y}_N) = \frac{1}{ND} \sum_{i=1}^N \sum_{j=1}^D (\hat{y}_{ij} - y_{ij})^2$$

donde $\hat{y}_{ij}$ es la predicción del modelo para la i -ésima muestra en la j -ésima dimensión y y_{ij} es el valor real. El cociente ND es la normalización de la suma tomando en cuenta la cantidad de datos (se ve en estadística). Por otro lado, la fórmula del gradiente se obtiene de las derivadas parciales con respecto a los valores del modelo:

$$\frac{\partial L}{\partial \hat{y}_{ij}} = \frac{2}{ND} (\hat{y}_{ij} - y_{ij})$$

ya que todo lo demás es constante con respecto a la variable que se está derivando.

Esta función se puede ver en ejemplos del mundo real como en la predicción de precios de viviendas o en la estimación de temperaturas futuras a partir de datos históricos.

8. BINARY CROSS ENTROPY LOSS

Fundamentación Matemática: La función de pérdida conocida como "Binary Cross Entropy" es el estándar al momento de lidiar con tareas de clasificación binaria. Esta se encarga de medir que el nivel de éxito con el que la distribución de probabilidad desarrollada por el modelo aproxima a la distribución empírica.

Algunas de sus características más importantes son que logra medir el concepto de entropía dentro de un conjunto de datos (conocido como "sorpresa") y proporciona gradientes con bastante información cuando una predicción es errónea. En general, esta función de pérdida es de gran relevancia debido a que muestra una manera de trabajar con problemas relacionados a clasificación binaria a su vez de que otorga una herramienta para manejar pérdidas basadas en probabilidades.

La función L de pérdida corresponde a:

$$L(p_1, p_2, \dots, p_N) = \frac{-1}{N} \sum_{i=1}^N [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]$$

Esta función tiene dos componentes. El primero de ellos es el producto entre la probabilidad empírica y_i y el logaritmo de la probabilidad obtenida por el modelo p_i . El segundo término es análogo al primero pero usando los complementos de ambas probabilidades. Por la manera en la que están diseñados ambos términos, se tiene que el modelo penaliza cuando las predicciones modeladas tienen una magnitud muy diferente a la empírica. Si ambas probabilidades coinciden, no hay pérdida. Si la probabilidad modelada coincide con el complemento de la probabilidad que quería modelar, significa que el error fue de gran magnitud, tanto que el error tiende a infinito. Para evitar problemas de desbordamiento, se añade un pequeño épsilon dentro de los logaritmos usados en la función.

Finalmente, la fórmula detrás del gradiente (dx) surge a partir de la definición de L mostrada anteriormente, pues dicha definición implica que:

$$\frac{\partial L}{\partial p_i} = \frac{\partial}{\partial p_i} \left[\frac{-1}{N} \sum_{i=1}^N [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)] \right]$$

Por materia vista en MA-0450, queda que:

$$\frac{\partial L}{\partial p_i} = \frac{-1}{N} \left[\frac{y_i}{p_i} - \frac{1 - y_i}{1 - p_i} \right]$$

Al operar la fracciones y simplificar, se obtiene que:

$$\frac{\partial L}{\partial p_i} = \frac{p_i - y_i}{N \cdot p_i(1 - p_i)}$$

Finalmente, al considerar $p_i(1 - p_i)$ lo suficientemente no pequeño, se deduce que:

$$\frac{\partial L}{\partial p_i} \approx \frac{p_i - y_i}{N}$$

Lo que coincide con la fórmula para el cálculo del gradiente brindada por el enunciado.

Esta función se puede ver en ejemplos del mundo real como en la detección de spam en un correo electrónico, o en el estudio de que tan probable es que una persona haga click en un anuncio de una página determinada.

9. CATEGORICAL CROSS ENTROPY BACKWARD

Fundamentación matemática: La propagación inversa de "Softmax + Categorical Cross Entropy" permite obtener el gradiente de la pérdida con respecto a los logits en un solo paso, combinando la derivada de Softmax y la derivada de la entropía cruzada gracias a la regla de la cadena.

Sea $p_i \in \mathbb{R}^C$ el vector de probabilidades obtenido por Softmax para la i -ésima muestra, y sea $e_i \in \{0, 1\}^C$ su representación one-hot. La pérdida para dicha muestra es:

$$\ell_i = - \sum_{k=1}^C e_{ik} \log(p_{ik})$$

Al derivar la composición Softmax- ℓ con respecto al logit z_{ij} , las derivadas de ambas funciones se cancelan parcialmente y queda la siguiente simplificación:

$$\frac{\partial \ell_i}{\partial z_{ij}} = p_{ij} - e_{ij}$$

Promediando sobre las N muestras del lote, el gradiente final es:

$$\frac{\partial L}{\partial z_{ij}} = \frac{p_{ij} - e_{ij}}{N}$$

lo cual coincide con la fórmula dada por el enunciado. Esta simplificación es la razón por la que, en la práctica, casi nunca se calculan por separado las derivadas de Softmax y de la entropía cruzada.

10. SGD OPTIMIZER

Se implementa la función `sgd_step(w, dw, lr)` que realiza un paso del descenso de gradiente estocástico

La regla de actualización es:

$$w_{t+1} = w_t - \eta \cdot \nabla L(w_t)$$

donde w_t son los pesos actuales, $\nabla L(w_t)$ es el gradiente de la pérdida con respecto a los pesos calculado mediante backpropagation, y η es la tasa de aprendizaje que controla el tamaño del paso. La idea detrás de esta fórmula es simple: el gradiente apunta hacia donde la pérdida *sube*, entonces nos movemos en la dirección contraria para *bajarla*.

La elección de η es crítica: un valor demasiado grande puede hacer que el modelo diverja, mientras que uno demasiado pequeño hace que el entrenamiento sea muy lento. Los valores típicos están entre 10^{-3} y 10^{-1} .

Esta función se puede ver en ejemplos del mundo real como en el entrenamiento inicial de modelos sencillos de clasificación, donde su simplicidad lo hace fácil de entender y depurar.

11. MOMENTUM OPTIMIZER

Fundamentación matemática: El optimizador con momento es una mejora del descenso de gradiente estocástico que agrega un término de "velocidad" acumulada. Esto permite acelerar el aprendizaje cuando los gradientes apuntan consistentemente en una misma dirección, y suaviza las oscilaciones cuando no.

Las reglas de actualización son:

$$v_{t+1} = \mu v_t + \nabla_w L$$

$$w_{t+1} = w_t - \eta v_{t+1}$$

donde μ es el coeficiente de momento, η es la tasa de aprendizaje y $\nabla_w L$ es el gradiente de la pérdida con respecto a los pesos. Cuando v no ha sido inicializado se toma $v_0 = \mathbf{0}$, por lo que el primer paso coincide con el descenso de gradiente normal.

Un valor de μ cercano a 1 le da más peso a los gradientes pasados, generando actualizaciones más suaves pero más lentas de reaccionar ante cambios bruscos. El valor estándar es $\mu = 0,9$.

Este método se puede ver en ejemplos del mundo real como en el entrenamiento de redes neuronales profundas para reconocimiento de imágenes, donde los gradientes suelen ser ruidosos.

12. RMSProp OPTIMIZER

Fundamentación matemática: RMSProp es un optimizador adaptativo que busca corregir el problema de usar una única tasa de aprendizaje global, como ocurre en el descenso de gradiente estocástico. Para esto, mantiene una media móvil de los cuadrados de los gradientes y la usa para escalar la actualización de cada parámetro de forma individual.

Las reglas de actualización son:

$$s_{t+1} = \rho s_t + (1 - \rho)(\nabla_w L)^2$$

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{s_{t+1}} + \varepsilon} \nabla_w L$$

donde ρ es el coeficiente de decaimiento (usualmente $\rho = 0,99$) y ε es una constante pequeña que evita la división entre cero. Las operaciones de cuadrado y raíz se hacen entrada por entrada. Si s no ha sido inicializado, se toma $s_0 = \mathbf{0}$.

La idea detrás de esto es que los parámetros con gradientes históricamente grandes reciben actualizaciones más pequeñas, y viceversa. Esto ayuda bastante cuando el paisaje de pérdida tiene curvaturas muy distintas entre dimensiones.

Este optimizador se puede ver en ejemplos del mundo real como en el entrenamiento de redes recurrentes para procesamiento de lenguaje natural, donde la escala de los gradientes cambia mucho entre capas.

13. ADAM OPTIMIZER

Fundamentación matemática: Adam combina las ideas del optimizador con momento y de RMSProp en un solo método. Mantiene una estimación del primer momento (la media) y del segundo momento (la varianza no centrada) de los gradientes.

Las reglas de actualización son:

$$m_{t+1} = \beta_1 m_t + (1 - \beta_1) \nabla_w L$$

$$v_{t+1} = \beta_2 v_t + (1 - \beta_2) (\nabla_w L)^2$$

donde β_1 y β_2 son los coeficientes de decaimiento de cada momento (usualmente $\beta_1 = 0,9$ y $\beta_2 = 0,999$). Como m y v se inicializan en cero, ambas estimaciones están sesgadas hacia cero en los primeros pasos. Para corregir esto se calculan las versiones corregidas:

$$\hat{m}_{t+1} = \frac{m_{t+1}}{1 - \beta_1^{t+1}}, \quad \hat{v}_{t+1} = \frac{v_{t+1}}{1 - \beta_2^{t+1}}$$

Finalmente, la actualización de los pesos queda:

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{\hat{v}_{t+1}} + \varepsilon} \hat{m}_{t+1}$$

La corrección de sesgo es lo que hace que las estimaciones sean confiables desde el primer paso, y es la razón por la que Adam funciona bien incluso sin mucho ajuste de hiperparámetros. Por esto mismo, es el optimizador más usado en la práctica, y se puede ver en ejemplos del mundo real como en el preentrenamiento de modelos de lenguaje de gran escala.

14. WEIGHT INITIALIZATION (XAVIER/GLOROT)

Se implementa la función `xavier_init(shape)` que inicializa los pesos de una capa usando la distribución uniforme de Xavier/Glorot.

El problema que resuelve esta técnica es que si los pesos se inicializan con valores muy grandes o muy pequeños, las activaciones y los gradientes pueden explotar o desvanecerse al propagarse por las capas. Xavier propone inicializar los pesos muestreando de una distribución uniforme:

$$W_{ij} \sim U \left(-\sqrt{\frac{6}{fan_{in} + fan_{out}}}, \sqrt{\frac{6}{fan_{in} + fan_{out}}} \right)$$

donde fan_{in} es el número de entradas a la capa y fan_{out} el número de salidas. El límite $\sqrt{\frac{6}{fan_{in} + fan_{out}}}$ está diseñado para que la varianza de las activaciones se mantenga aproximadamente constante a lo largo de las capas, evitando que la información se amplifique o se apague progresivamente.

Esta inicialización es especialmente adecuada para funciones de activación simétricas alrededor del cero como $\tanh$ o sigmoide, y se puede ver en ejemplos del mundo real como en la inicialización de redes profundas para procesamiento de lenguaje natural.

15. WEIGHT INITIALIZATION (KAIMING/HE)

Se implementa la función `kaiming_init(shape)` que inicializa los pesos usando la distribución normal de Kaiming/He, diseñada específicamente para capas con activación ReLU.

La diferencia con Xavier es que ReLU elimina aproximadamente la mitad de las activaciones (las negativas), lo que reduce la varianza a la mitad en cada capa. Para compensar esto, Kaiming propone usar una varianza mayor, muestreando de una distribución normal:

$$W_{ij} \sim \mathcal{N}\left(0, \sqrt{\frac{2}{fan_{in}}}\right)$$

El factor 2 en el numerador compensa exactamente la reducción de varianza que produce ReLU. Nótese que a diferencia de Xavier, aquí solo importa fan_{in} y no fan_{out} , pues la corrección apunta a mantener la varianza estable en el paso hacia adelante.

16. DROPOUT FORWARD (INVERTED)

La técnica de regularización *Dropout* es una herramienta sumamente poderosa que permite la prevención de la sobrecapacitación de ciertas neuronas. Esto logra al eliminar ciertas neuronas de manera aleatoria durante el entrenamiento. Pese a lo poco intuitivo que parece, este proceso ayuda a que la red no depende de una neurona en específico al momento de realizar alguna tarea. En general, esto ayuda a que varias neuronas sean capaces de hacer el mismo trabajo de manera relativamente independiente.

La implementación estándar de la técnica *Dropout*, en la práctica, es la conocida como *Inverted Dropout*. Esto último se debe a que, en el método *Dropout*, el valor esperado del tensor original es distinto al del tensor modificado, pues sus esperanzas satisfacen que:

$$\mathbb{E}[y] = (1 - p)\mathbb{E}[x],$$

donde y es el tensor modificado y x el original. La igualdad anterior se puede mostrar usando resultados vistos en CA-0721. Sin embargo, *Inverted Dropout* corrige esto último al tomar el tensor $y^* := y(1 - p)^{-1}$, donde y es el tensor obtenido con el método normal. Así, por propiedades de la esperanza, se tendría que la esperanza de y^* y x son iguales.

En general, esta técnica no solo es simple sino que su impacto es muy significativo, pues permite "promediar" las neuronas de la red. lo anterior a su vez explica el motivo de que sea tan popular, pues está presente en la mayoría de arquitecturas de redes neuronales. Además, la intensidad de este tipo de entrenamiento se puede medir mediante la variación del parámetro p . A mayor p , más neuronas serán eliminadas, mientras que un p pequeño producirá el efecto contrario. Por esto mismo, el estándar es tomar $p = 0,5$.

17. DROPOUT BACKWARD

La propagación inversa mediante la técnica *Dropout* se encarga de regular las actualizaciones de los gradientes, pues si una neurona es desechada durante el entrenamiento entonces se espera que los gradientes no "fluyan" en neuronas "muertas". Todo el proceso anterior es computacionalmente eficiente y, al mismo tiempo, es fundamental para que las redes neuronales funcionen de la mejor manera. Es justamente por esto que la función vista en el ejercicio 16 almacenaba el tensor *mask* en su output, pues es necesario reutilizarlo en esta función para saber por donde deben "moverse" los gradientes.

A su vez, por todo lo explicado en el ejercicio 16, los gradientes se van a escalar por un factor de $(1 - p)^{-1}$ al pasar por las neuronas.

Este proceso retoma lo visto en el ejercicio 4 y lo utiliza para complementar lo visto en el ejercicio 16, por lo que es de vital relevancia en el entrenamiento de las redes neuronales en la gran mayoría de contextos. Es por esto último que las aplicaciones vistas en el ejercicio 16 aplican de igual manera para lo visto en el presente ejercicio.

18. BATCH NORMALIZATION FORWARD (TRAIN VS EVAL)

Se implementa la función `batchnorm_forward` que normaliza cada feature del batch durante el entrenamiento.

Se calculan sobre todas las muestras del batch para cada feature j :

$$\mu_j = \frac{1}{N} \sum_{i=1}^N x_{ij} \quad \sigma_j^2 = \frac{1}{N} \sum_{i=1}^N (x_{ij} - \mu_j)^2$$

Cada valor se normaliza usando la media y varianza de su feature:

$$\hat{x}_{ij} = \frac{x_{ij} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$

Se aplican los parámetros aprendibles γ y β , uno por feature:

$$y_{ij} = \gamma_j \hat{x}_{ij} + \beta_j$$

Durante el entrenamiento se mantienen promedios móviles para usarse en inferencia:

$$\begin{aligned} \mu_{\text{running}} &\leftarrow \alpha \cdot \mu_{\text{running}} + (1 - \alpha) \cdot \mu_{\text{batch}} \\ \sigma_{\text{running}}^2 &\leftarrow \alpha \cdot \sigma_{\text{running}}^2 + (1 - \alpha) \cdot \sigma_{\text{batch}}^2 \end{aligned}$$

En lugar de calcular estadísticas del batch actual, se usan directamente las acumuladas durante el entrenamiento:

$$\hat{x}_{ij} = \frac{x_{ij} - \mu_{\text{running}}}{\sqrt{\sigma_{\text{running}}^2 + \varepsilon}}$$

19. BATCH NORMALIZATION BACKWARD

Se implementa la función `batchnorm_backward(dout, cache)` que calcula los gradientes de la pérdida respecto a cada parámetro de la capa de normalización, usando la regla de la cadena.

Como β se suma a cada muestra del batch, su gradiente acumula `dout` sobre todas las muestras:

$$\frac{\partial L}{\partial \beta_j} = \sum_{i=1}^N \text{dout}_{ij}$$

Gradiente respecto a γ :

Como γ multiplica a $\hat{x}$, por la regla del producto su gradiente es:

$$\frac{\partial L}{\partial \gamma_j} = \sum_{i=1}^N \text{dout}_{ij} \cdot \hat{x}_{ij}$$

Gradiente respecto a X :

Es el más complejo, ya que x afecta la salida tanto directamente a través de $\hat{x}$, como indirectamente a través de μ y σ^2 . Primero se obtiene el gradiente que llega a $\hat{x}$:

$$\frac{\partial L}{\partial \hat{x}} = \text{dout} \cdot \gamma$$

Luego se corrige el efecto de la media y la varianza, que también dependen de x :

$$\delta \mu = \frac{1}{N} \sum_{i=1}^N \frac{\partial L}{\partial \hat{x}_{ij}}$$

$$\delta \sigma^2 = \frac{1}{N} \sum_{i=1}^N \frac{\partial L}{\partial \hat{x}_{ij}} \cdot (x_{ij} - \mu_j)$$

Combinando las tres correcciones:

$$\frac{\partial L}{\partial x_{ij}} = \frac{1}{\sqrt{\sigma_j^2 + \varepsilon}} \left(\frac{\partial L}{\partial \hat{x}_{ij}} - \delta \mu_j - \hat{x}_{ij} \cdot \delta \sigma_j^2 \right)$$

20. LAYER NORMALIZATION FORWARD & BACKWARD

Se implementan las funciones `layernorm_forward` y `layernorm_backward`, análogas a los ejercicios 18 y 19 pero con una diferencia clave: en lugar de normalizar por feature sobre el batch (axis=0), se normaliza por muestra sobre las features (axis=1).

A diferencia de BatchNorm donde μ_j y σ_j^2 son por feature, aquí se calculan por cada muestra i :

$$\mu_i = \frac{1}{D} \sum_{j=1}^D x_{ij} \quad \sigma_i^2 = \frac{1}{D} \sum_{j=1}^D (x_{ij} - \mu_i)^2$$

Ambas tienen forma $(N, 1)$ con `keepdims=True` para que el broadcasting funcione correctamente sobre (N, D) .

$$\hat{x}_{ij} = \frac{x_{ij} - \mu_i}{\sqrt{\sigma_i^2 + \varepsilon}}$$

Igual que en BatchNorm, se aplican los parámetros aprendibles γ y β :

$$y_{ij} = \gamma_j \hat{x}_{ij} + \beta_j$$

Los gradientes de β y γ se calculan igual que en el ejercicio 19, sumando sobre el batch:

$$\frac{\partial L}{\partial \beta_j} = \sum_{i=1}^N \text{dout}_{ij} \quad \frac{\partial L}{\partial \gamma_j} = \sum_{i=1}^N \text{dout}_{ij} \cdot \hat{x}_{ij}$$

Para dx , las correcciones por media y varianza ahora promedian sobre las features (axis=1) en lugar del batch, manteniendo forma $(N, 1)$:

$$\delta \mu_i = \frac{1}{D} \sum_{j=1}^D \frac{\partial L}{\partial \hat{x}_{ij}} \quad \delta \sigma_i^2 = \frac{1}{D} \sum_{j=1}^D \frac{\partial L}{\partial \hat{x}_{ij}} \cdot (x_{ij} - \mu_i)$$

El gradiente final tiene la misma forma que en el ejercicio 19:

$$\frac{\partial L}{\partial x_{ij}} = \frac{1}{\sqrt{\sigma_i^2 + \varepsilon}} \left(\frac{\partial L}{\partial \hat{x}_{ij}} - \delta \mu_i - \hat{x}_{ij} \cdot \delta \sigma_i^2 \right)$$